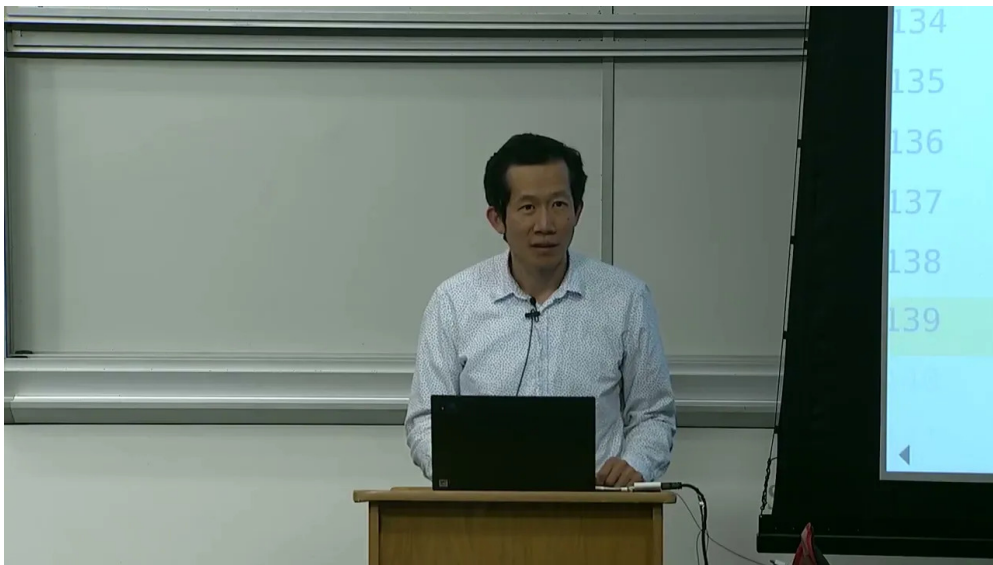


课程笔记

# CS336 Lecture 10: Inference

基于 Percy Liang 授课内容整理

2025 年 5 月



视频作者/频道: Stanford Online

发布日期: 2025

视频时长: 1:22:48

视频链接: <https://www.youtube.com/watch?v=fcgPYo30tV0>

# 目录

|                            |           |
|----------------------------|-----------|
| <b>1 引言：推理的重要性</b>         | <b>2</b>  |
| 1.1 推理的应用场景                | 2         |
| 1.2 推理的核心度量指标              | 2         |
| 1.3 推理与训练的核心差异             | 3         |
| 1.4 推理优化技术全景               | 3         |
| 1.5 推理生态                   | 3         |
| 1.6 本章小结                   | 4         |
| <b>2 理解推理工作负载</b>          | <b>4</b>  |
| 2.1 回顾：算术强度                | 4         |
| 2.2 朴素推理与 KV Cache         | 6         |
| 2.3 推理的两个阶段                | 7         |
| 2.4 MLP 层的算术强度分析           | 8         |
| 2.5 注意力层的算术强度分析            | 8         |
| 2.6 延迟与吞吐量的理论分析            | 8         |
| 2.7 本章小结                   | 10        |
| <b>3 有损优化：缩减 KV Cache</b>  | <b>10</b> |
| 3.1 分组查询注意力 (GQA)          | 10        |
| 3.2 多头潜在注意力 (MLA)          | 11        |
| 3.3 跨层注意力 (CLA)            | 12        |
| 3.4 局部注意力                  | 12        |
| 3.5 本章小结                   | 12        |
| <b>4 Transformer 的替代架构</b> | <b>13</b> |
| 4.1 状态空间模型 (SSM)           | 13        |
| 4.1.1 Mamba 的核心机制：输入依赖选择   | 14        |
| 4.1.2 Jamba：混合架构的设计权衡      | 14        |
| 4.2 扩散模型                   | 15        |
| 4.3 本章小结                   | 16        |
| <b>5 量化与模型剪枝</b>           | <b>16</b> |
| 5.1 量化：降低数值精度              | 16        |
| 5.1.1 LLM.int8()：处理异常值     | 17        |
| 5.1.2 AWQ：激活感知量化           | 17        |
| 5.2 模型剪枝与知识蒸馏              | 18        |
| 5.3 本章小结                   | 18        |
| <b>6 无损加速：投机采样</b>         | <b>18</b> |
| 6.1 核心直觉：验证比生成快            | 18        |
| 6.2 投机采样算法                 | 19        |
| 6.3 正确性保证                  | 19        |
| 6.4 实践配置与扩展                | 20        |

|          |                             |           |
|----------|-----------------------------|-----------|
| 6.5      | 本章小结                        | 21        |
| <b>7</b> | <b>处理动态工作负载</b>             | <b>21</b> |
| 7.1      | 连续批处理 (Continuous Batching) | 21        |
| 7.2      | PagedAttention 与 vLLM       | 22        |
| 7.3      | 本章小结                        | 23        |
| <b>8</b> | <b>总结与延伸</b>                | <b>23</b> |
| 8.1      | 核心要点回顾                      | 23        |
| 8.2      | 关键洞察                        | 23        |
| 8.3      | 拓展阅读                        | 24        |

# 1 引言：推理的重要性

本节课的主题是**推理** (Inference)：给定一个已经训练好的固定模型，根据输入的提示 (prompt) 生成响应。推理看似简单，但其效率优化涉及硬件特性、算法设计和系统工程等多个层面，是大语言模型实际应用中最关键的环节之一。

## 推理的定义

**推理** (Inference)：给定一个**固定模型**，根据提示生成响应。与训练不同，推理不更新模型参数，但需要反复执行，因此效率至关重要。

## 1.1 推理的应用场景

推理不仅仅是“搭建一个聊天机器人”那么简单，它在以下多个场景中扮演核心角色：

1. **实际使用**：聊天机器人、代码补全（如 Cursor）、批量数据处理等
2. **模型评估**：在指令遵循等任务上评估模型性能，本质上也是推理
3. **测试时计算** (Test-time Compute)：让模型“思考”更多步骤再输出最终答案——思考本身就是生成 token
4. **强化学习训练**：采样响应并根据奖励评分，采样过程即推理

## 推理的规模有多大？

一些数据帮助理解推理的规模：

- OpenAI 每天生成超过 1000 亿个词
- Cursor 每天生成约 10 亿行被用户接受的代码

训练是一次性成本，而推理会被反复执行无数次，因此推理成本在总成本中的占比日益增长。

## 1.2 推理的核心度量指标

### 推理性能的三个核心指标

- **首 token 延迟** (TTFT, Time-to-First-Token)：用户等待第一个生成 token 出现的时间。对交互式应用至关重要
- **延迟** (Latency, 秒/token)：每个 token 的生成速度。影响用户的流式阅读体验
- **吞吐量** (Throughput, tokens/秒)：系统整体每秒生成的 token 数。对批处理应用尤为重要

### 高吞吐量不等于低延迟

吞吐量衡量的是系统整体的处理能力，而延迟关注的是单个用户的体验。一个系统可以有非常高的吞吐量（同时处理大量请求），但某些请求的延迟可能很高。这两个指标之间存在根本性的权衡。

### 1.3 推理与训练的核心差异

- **训练**: 可以看到所有 token, 可以在序列维度上并行化 (矩阵乘法), **计算受限** (compute-limited)
- **推理**: 必须逐 token 顺序生成 (每个 token 依赖之前的所有 token), 难以充分利用计算资源, **内存受限** (memory-limited)

这一根本差异决定了推理优化的方向: 与训练追求更高计算利用率不同, 推理优化的核心是减少内存传输。

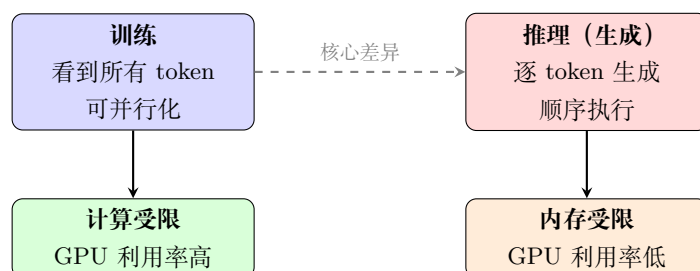


图 1: 训练与推理的核心差异: 计算受限 vs. 内存受限

### 1.4 推理优化技术全景

本节课将覆盖以下推理优化技术, 按照从理解问题到解决问题的顺序展开:

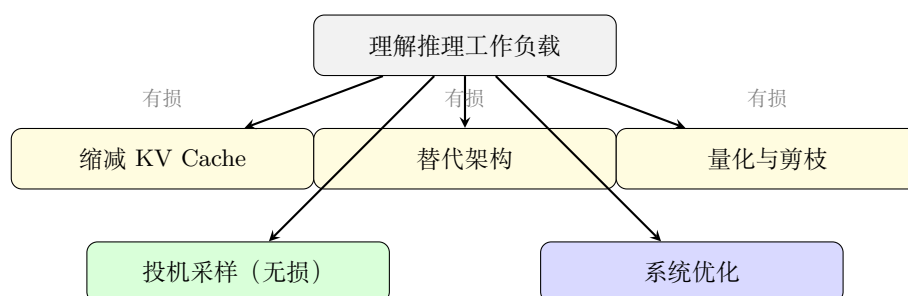


图 2: 本节课推理优化技术的组织结构

### 1.5 推理生态

从事推理优化的公司和项目包括:

- **闭源模型提供商**: OpenAI、Anthropic、Google 等
- **开源权重模型服务商**: Together、Fireworks、DeepInfra 等
- **开源推理框架**:
  - vLLM (UC Berkeley)
  - TensorRT-LLM (NVIDIA)
  - TGI (Hugging Face)

## 1.6 本章小结

推理是大语言模型从训练到实际部署的关键环节。它不仅出现在最终用户交互中，还贯穿于模型评估、测试时计算和强化学习训练的全过程。推理的核心特征是**顺序生成**和**内存受限**，这决定了后续所有优化技术的方向。

## 2 理解推理工作负载

要优化推理，首先需要深入理解推理过程中的计算和内存传输特性。本节从矩阵乘法的算术强度出发，分析 Transformer 推理中 MLP 层和注意力层的性能特征。

### 2.1 回顾：算术强度

#### 符号约定

本节使用的符号（与 Scaling Book 一致）：

- $B$ : 批大小 (batch size)
- $S$ : 条件序列长度 (prompt 长度)
- $T$ : 生成序列长度
- $D$ : 模型隐藏维度
- $F$ : MLP 隐藏维度 (通常  $F = 4D$ )
- $N$ : Query 注意力头数
- $K$ : Key/Value 注意力头数
- $H$ : 每个头的维度 ( $N \times H = D$ )
- $L$ : 层数
- $V$ : 词表大小

**算术强度** (Arithmetic Intensity) 是衡量计算效率的核心指标：每传输一个字节数据能完成多少次浮点运算。

以基本的矩阵乘法  $Y = X \cdot W$  为例，其中  $X$  的形状为  $B \times D$ ， $W$  的形状为  $D \times F$ ：

1. 从 HBM 读取  $X$ ：传输  $2BD$  字节 (bf16)
2. 从 HBM 读取  $W$ ：传输  $2DF$  字节
3. 计算  $Y = X \cdot W$ ： $2BDF$  FLOPs
4. 将  $Y$  写回 HBM：传输  $2BF$  字节

$$\text{算术强度} = \frac{\text{FLOPs}}{\text{bytes transferred}} = \frac{2BDF}{2BD + 2DF + 2BF}$$

当  $B \ll D, F$  时（这是推理中的常见情况），可简化为：

算术强度  $\approx B$

### Worked Example: Llama 3 405B 的算术强度

以 Llama 3 405B 的第一个 MLP 层为例，权重矩阵  $W_{up}$  的形状为  $D \times F = 16384 \times 53248$  (bf16)。

**情况 1:**  $B = 1$  (单请求生成)

$$\text{算术强度} = \frac{2 \times 1 \times 16384 \times 53248}{2 \times 1 \times 16384 + 2 \times 16384 \times 53248 + 2 \times 1 \times 53248} \approx \frac{1.74 \times 10^9}{1.74 \times 10^9} \approx 1.0$$

GPU 几乎所有时间在等内存传输。

**情况 2:**  $B = 256$  (大批量)

算术强度  $\approx B = 256$

仍低于 H100 的加速器强度 295，依然处于内存受限区域——但已接近拐点。

**结论:** 即使是大批量推理，405B 模型在 H100 上仍然很难达到计算受限。

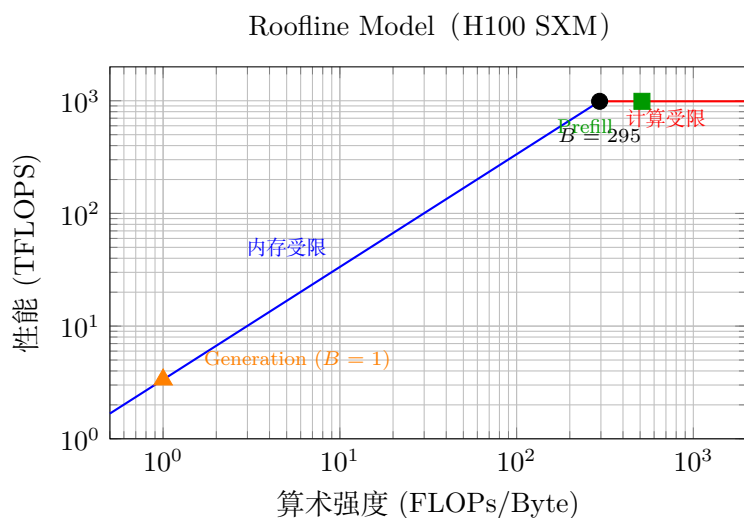


图 3: H100 的 Roofline 模型。Generation 阶段 ( $B = 1$ ) 位于内存受限区域的最低端，Prefill 则可以达到计算受限区域。

## H100 的加速器强度

对于 H100 GPU:

- bf16 峰值算力: 989 TFLOPS
- HBM 带宽: 3.35 TB/s
- 加速器强度 =  $989/3.35 \approx 295$

因此:

- 当  $B > 295$  时: **计算受限** (compute-limited) ——GPU 利用率高, 效率好
- 当  $B < 295$  时: **内存受限** (memory-limited) ——GPU 大部分时间在等待数据传输

## B

当  $B = 1$  时 (矩阵-向量乘法), 算术强度仅为 1。这意味着每读取一个权重元素只做一次乘法运算——几乎所有时间都花在内存读取上。**这正是自回归生成的典型场景。**

## 2.2 朴素推理与 KV Cache

**朴素推理:** 生成每个 token 时, 将整个历史序列送入 Transformer。生成  $T$  个 token 需要  $O(T^3)$  FLOPs (每次前向传播  $O(T^2)$ )。

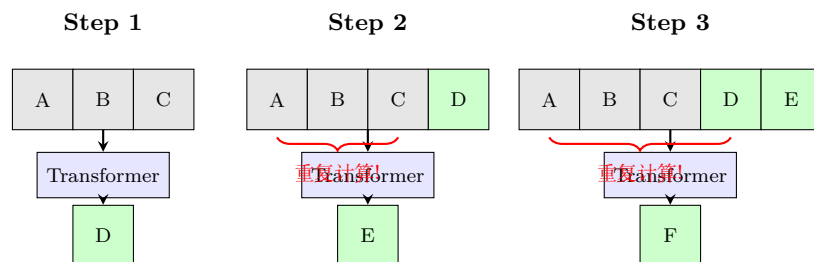


图 4: 朴素推理: 每步都重新处理整个序列, 灰色为 prompt, 绿色为已生成 token。前缀部分的计算完全冗余。

### 观察: 前缀计算的冗余

在因果 (自回归) Transformer 中, 由于 causal mask 的存在, 生成第  $t$  个 token 时, 前  $t-1$  个 token 的中间表示与之前完全相同。这些计算是完全冗余的。注意: 这一性质仅对单向 (causal) Transformer 成立。对于双向 Transformer (如 BERT), 新增 token 会改变所有位置的表示。

**解决方案: KV Cache**——将每一层注意力计算中的 Key 和 Value 向量缓存在 HBM 中, 后续生成时直接复用。



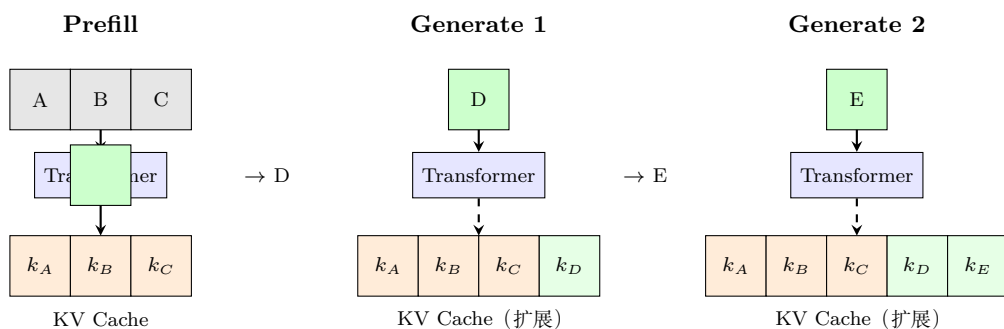


图 5: KV Cache 推理: Prefill 阶段并行处理 prompt 并建立缓存, Generation 阶段每步只处理一个新 token, 复用缓存中的 K/V 向量。

KV Cache 的大小: 对于每个序列、每个 token、每一层、每个 KV 头, 存储一个  $H$  维向量:

$$\text{KV Cache 大小 (每序列)} = S \times K \times H \times L \times 2 \times 2 \text{ 字节}$$

其中第一个  $\times 2$  是 Key + Value, 第二个  $\times 2$  是 bf16 的字节数。

#### Worked Example: Llama 2 70B 的 KV Cache 大小

Llama 2 70B 参数:  $K = 8$  (GQA),  $H = 128$ ,  $L = 80$ ,  $S = 4096$ 。

单序列 KV Cache:

$$4096 \times 8 \times 128 \times 80 \times 2 \times 2 = 671,088,640 \text{ 字节} \approx 640 \text{ MB}$$

不同批大小下的总 KV Cache:

- $B = 1$ : 640 MB
- $B = 16$ :  $640 \times 16 = 10,240 \text{ MB} \approx 10 \text{ GB}$
- $B = 128$ :  $640 \times 128 = 81,920 \text{ MB} \approx 80 \text{ GB}$  (已占满 H100 的全部 HBM!)

加上模型参数本身约 140 GB (bf16),  $B = 128$  时总内存需求约 220 GB, 需要至少 3 张 H100。可见 KV Cache 是推理内存的主要瓶颈之一。

使用 KV Cache 后, 生成  $T$  个 token 的复杂度从  $O(T^3)$  降低到  $O(T^2)$ ——每步只需要  $O(T)$  的计算 (与缓存中的所有 Key 做点积)。

## 2.3 推理的两个阶段

### Prefill 与 Generation

1. **Prefill** (预填充): 给定 prompt, 将其编码为向量并填充 KV Cache。可以并行处理所有 token (类似训练), **计算受限**
2. **Generation** (生成): 逐个生成新的响应 token。每步只处理一个 token ( $T = 1$ ), **内存受限**

## 2.4 MLP 层的算术强度分析

对 MLP 层的矩阵乘法进行精确计量（使用 gated MLP：上投影  $W_{up}$ 、门控  $W_{gate}$ 、下投影  $W_{down}$ ）：

- **FLOPs**:  $6 \cdot B \cdot T \cdot D \cdot F$ （三个矩阵乘法，各贡献  $2BTDF$ ）
- **传输字节**:  $4BTD + 4BTF + 6DF$

当  $BT \ll D, F$  时（推理常见情况），MLP 层的算术强度简化为：

$$\text{MLP 算术强度} \approx B \cdot T$$

- **Prefill** ( $T = S$ ):  $BT$  可以很大，容易达到计算受限
- **Generation** ( $T = 1$ ): 算术强度仅为  $B$ ——需要大量并发请求才能饱和 GPU

## 2.5 注意力层的算术强度分析

使用 FlashAttention 时，注意力层的计量：

- **FLOPs**:  $4BSTD$
- **传输字节**:  $4BSD + 4BTD$

$$\text{注意力算术强度} = \frac{ST}{S+T}$$

代入两个阶段：

- **Prefill** ( $T = S$ ): 强度为  $S/2$ ，随序列长度增长
- **Generation** ( $T = 1$ ): 强度为  $\frac{S}{S+1} < 1$ ——**始终内存受限！**

### 注意力层与 MLP 层的关键区别

- **MLP 层**：所有序列共享相同的权重矩阵（ $W_{up}$ ,  $W_{gate}$ ,  $W_{down}$  不依赖于  $B$ ），因此增大批大小可以分摊权重读取成本
- **注意力层**：每个序列有自己的 KV Cache（Q、K、V 都依赖于  $B$ ），因此增大批大小无法提高算术强度

这意味着注意力层在生成阶段不可避免地是内存受限的。

## 2.6 延迟与吞吐量的理论分析

既然推理是内存受限的，性能瓶颈在于内存传输速度。以 Llama 2 13B 在单个 H100 上的推理为例：

模型配置：  $S = 1024$ ,  $D = 5120$ ,  $F = 13824$ ,  $N = 40$ ,  $K = 40$ ,  $H = 128$ ,  $L = 40$ ,  $V = 32000$ 。

- **参数数量**：约 130 亿

- **参数存储** (bf16):  $\text{num\_params} \times 2$  字节
- **KV Cache** (每序列):  $S \times K \times H \times L \times 2 \times 2$  字节
- **总内存** =  $B \times \text{KV Cache} + \text{参数存储}$

延迟 (每个 token 的生成时间) 由内存传输决定:

$$\text{Latency} = \frac{\text{Total Memory}}{\text{Memory Bandwidth}}$$

$$\text{Throughput} = \frac{B}{\text{Latency}}$$

不同批大小下的表现:

| $B$ | 内存占用    | 延迟 (ms/token) | 吞吐量 (tokens/s) |
|-----|---------|---------------|----------------|
| 1   | ~26 GB  | ~8            | ~124           |
| 64  | ~79 GB  | ~24           | ~2700          |
| 256 | ~240 GB | —             | —              |

表 1: Llama 2 13B 在 H100 上不同批大小下的推理性能 (理论值)

### 延迟与吞吐量的权衡

- 小批大小 → 低延迟, 低吞吐量
- 大批大小 → 高延迟, 高吞吐量 (但有上限, 且受内存容量限制)
- $B = 256$  时, Llama 2 13B 已超出 H100 的 80GB HBM 容量

**并行策略:**

- **简单并行:** 启动  $M$  个模型副本, 延迟不变, 吞吐量提升  $M$  倍
- **模型并行:** 将模型和 KV Cache 分片到多个 GPU 上 (更复杂, 但能处理单卡放不下的模型)

### TTFT 与 Prefill 的关系

首 token 延迟 (TTFT) 本质上由 Prefill 阶段决定。实际部署中可以采用分离策略:

- Prefill 阶段使用较小的批大小, 以缩短 TTFT
- Generation 阶段使用较大的批大小, 以提高吞吐量

### Prefill 与 Generation 的资源竞争

在同一 GPU 上混合处理 Prefill 和 Generation 请求时, 两者会竞争计算资源和内存带宽。一个长 prompt 的 Prefill 可能阻塞所有正在进行的 Generation 请求。生产环境中的解决方案包括:

- **Prefill/Generation 分离:** 使用不同的 GPU 池分别处理两个阶段
- **分块 Prefill (Chunked Prefill):** 将长 prompt 切成小块, 与 Generation 交替执行

## 2.7 本章小结

推理工作负载的核心特征是：

- Prefill 是计算受限的（类似训练），Generation 是内存受限的
- MLP 层的算术强度为  $B \cdot T$ ，可通过增大批大小改善
- 注意力层的算术强度在生成时  $< 1$ ，无法通过增大批大小改善
- 延迟和吞吐量之间存在根本性权衡，受 GPU 内存容量约束

## 3 有损优化：缩减 KV Cache

既然推理是内存受限的，核心优化思路就是**减少需要传输的内存量**。KV Cache 是推理中最大的内存消耗之一，本节介绍几种在不显著损失精度的前提下缩减 KV Cache 的方法。

### 3.1 分组查询注意力 (GQA)

#### GQA 的核心思想

标准多头注意力 (MHA) 中，每个 Query 头都有对应的 Key 和 Value 头 ( $K = N$ )。GQA 让多个 Query 头**共享**同一组 Key/Value 头，从而减少 KV Cache 大小。

- **MHA**:  $K = N$  (每个 Query 头一个 KV 头)
- **MQA**:  $K = 1$  (所有 Query 头共享一个 KV 头)
- **GQA**:  $1 < K < N$  (折中方案)

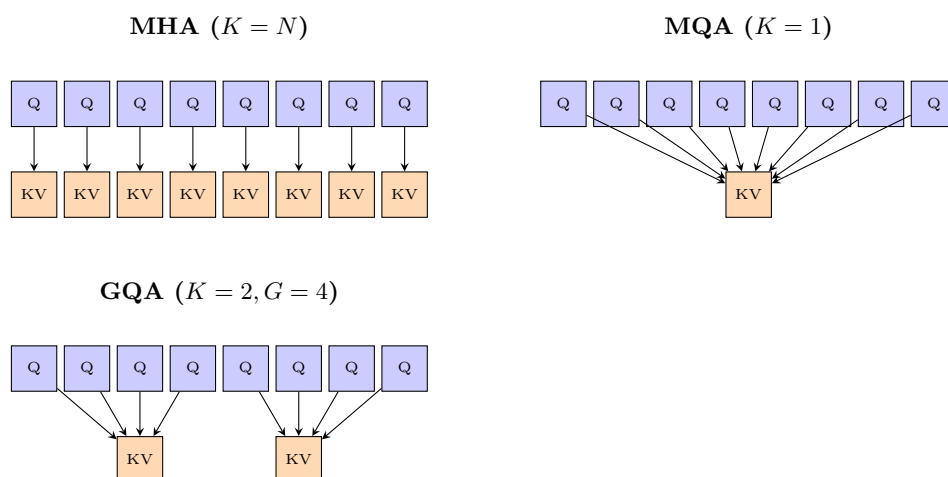


图 6: MHA、GQA、MQA 的区别: Q 头 (蓝) 到 KV 头 (橙) 的映射关系。GQA 是 MHA 和 MQA 之间的折中。

GQA 将 KV Cache 缩减为原来的  $K/N$  倍。以 Llama 2 13B 为例：

- 原始配置 ( $K = 40, B = 64$ ): 内存约 79 GB,  $B = 256$  时超出 H100 容量

- 使用 GQA ( $K = 8, B = 64$ ): 内存显著减少, 吞吐量大幅提升
- 使用 GQA ( $K = 8, B = 256$ ): 现在可以放入 H100 内存, 进一步提升吞吐量

| 模型                 | $N$ | $K$ | KV Cache/序列 | 压缩比 | $B_{\max}$ (H100) |
|--------------------|-----|-----|-------------|-----|-------------------|
| Llama 2 7B (MHA)   | 32  | 32  | 1024 MB     | 1×  | ~50               |
| Llama 2 70B (GQA)  | 64  | 8   | 640 MB      | 8×  | ~100              |
| Llama 3 8B (GQA)   | 32  | 8   | 256 MB      | 4×  | ~200              |
| Llama 3 70B (GQA)  | 64  | 8   | 640 MB      | 8×  | ~100              |
| Llama 3 405B (GQA) | 128 | 8   | 1280 MB     | 16× | —                 |

表 2: 不同 Llama 模型的 KV Cache 大小对比 ( $S = 4096, \text{bf16}$ )。  $B_{\max}$  为单 H100 GPU (80GB) 上大致可容纳的最大批大小 (需同时装下模型参数)。

### GQA 压缩比 $\neq$ 推理加速比

GQA 的  $K/N$  压缩比仅适用于 KV Cache 部分。模型参数大小不变, MLP 层的内存传输量也不变。实际加速比取决于 KV Cache 在总内存传输中的占比——序列越长、KV Cache 占比越大, GQA 的加速效果越明显。

### GQA 的精度影响

实验表明, GQA 在大幅降低推理成本的同时, 对模型精度的影响微乎其微。Llama 3 已全面采用 GQA。实际上 Llama 2 的 70B 版本已经使用了 GQA, 但较小的模型没有使用。

## 3.2 多头潜在注意力 (MLA)

MLA (Multi-head Latent Attention) 来自 DeepSeek v2, 采用与 GQA 不同的策略来缩减 KV Cache。

### MLA 的核心思想

GQA 通过减少 KV 头的数量来缩减缓存。MLA 则保持头的数量不变, 而是将每个 Key/-Value 向量投影到低维空间。

具体做法: 将每个 token 的 KV 表示从  $N \times H$  维投影到  $C$  维 ( $C \ll N \times H$ )。

DeepSeek v2 的配置:

- 原始 KV 维度:  $N \times H = 16384$
- 投影后维度:  $C = 512$  (压缩比超过 30 倍)
- 由于 MLA 与 RoPE 不兼容, 需要额外 64 维来编码位置信息, 实际为 576 维

### MLA 的精度表现

DeepSeek 的实验表明：

1. MHA 优于 GQA（精度更高，但开销也更大）
2. MLA 比 MHA 精度略好，同时推理开销远低于 MHA

MLA 在精度-效率的帕累托前沿上优于 GQA 和 MHA。

### MLA 与 RoPE 的兼容性问题

MLA 将 KV 向量投影到低维空间后再存储。但 RoPE 位置编码作用于原始 Key 向量——如果先投影再加 RoPE，则无法从低维表示恢复带位置信息的 Key。DeepSeek v2 的解决方案是为每个 token 额外存储 64 维的“解耦 RoPE Key”，使实际缓存维度从 512 增加到 576。这一工程细节在实现 MLA 时容易被忽略。

## 3.3 跨层注意力 (CLA)

### CLA 的核心思想

GQA 在头的维度上共享 KV 向量，CLA 则在层的维度上共享——让多个相邻层共享同一组 KV Cache。

实验表明，CLA 能够在精度与 KV Cache 大小之间改善帕累托前沿。CLA 可以与 GQA 叠加使用，进一步缩减缓存。

## 3.4 局部注意力

### 局部注意力 (滑动窗口注意力)

核心思想：每个 token 只关注局部上下文（最近的  $w$  个 token），而非全部历史。

- 有效上下文随层数线性增长
- KV Cache 大小不再依赖序列长度——变为常数

### 局部注意力的精度损失

纯局部注意力会损失对远距离依赖的建模能力。解决方案：**混合架构**——交替使用局部注意力层和全局注意力层。例如 character.ai 每 6 层使用 1 层全局注意力（结合 CLA），在大幅降低推理成本的同时保持了良好的精度。Mistral 7B 也采用了滑动窗口注意力。

## 3.5 本章小结

缩减 KV Cache 的核心策略包括：

1. **减少 KV 头数量**：GQA（头间共享）
2. **降低 KV 维度**：MLA（低秩投影）

- 3. **跨层共享**: CLA (层间共享)
- 4. **限制注意力范围**: 局部注意力 (固定窗口)

这些方法可以组合使用，且实验表明对精度的影响可控。所有方法都直接转化为更低的延迟和更高的吞吐量，因为推理是内存受限的。

| 方法    | 压缩维度      | 典型压缩比         | 精度影响      | 代表模型           |
|-------|-----------|---------------|-----------|----------------|
| GQA   | 头间共享      | 4-16×         | 极小        | Llama 2/3      |
| MQA   | 头间共享 (极端) | $N\times$     | 小-中等      | PaLM, Gemini   |
| MLA   | 低秩投影      | $>30\times$   | 极小 (甚至提升) | DeepSeek v2/v3 |
| CLA   | 层间共享      | 2-4×          | 小         | character.ai   |
| 局部注意力 | 限制序列范围    | $\propto w/S$ | 中等 (需混合)  | Mistral 7B     |

表 3: KV Cache 优化方法量化对比。压缩比为相对于标准 MHA 的缩减倍数。这些方法可以叠加使用。

## 4 Transformer 的替代架构

前一节在 Transformer 框架内进行优化。本节探讨更激进的方向：是否可以跳出 Transformer，从根本上改变推理的计算特性？

### 为什么需要新架构？

Transformer 的设计初衷是高效**训练**（通过并行处理序列），但自回归生成 + 全注意力的组合导致了推理的内存瓶颈。注意力层在生成阶段的算术强度  $< 1$ ，这是**架构层面的根本限制**，无法通过系统优化完全克服。

### 4.1 状态空间模型 (SSM)

状态空间模型借鉴信号处理和控制论的思想，试图用**次二次时间复杂度**处理长序列。

发展脉络：

1. **S4** (2021)：基于经典线性动力系统，具有 RNN 和卷积两种解释方式。在合成长序列任务上表现出色，但在语言建模上效果不佳
2. **问题诊断**：SSM 在**关联检索** (Associative Recall) 任务上表现差——给定一系列键值对，查找特定键对应的值。这类任务需要“精确查找”能力，而 SSM 更擅长平滑的信号处理
3. **Mamba** (2023)：让 SSM 参数依赖于输入 (input-dependent)，在 1B 规模上匹配 Transformer 性能
4. **Jamba** (AI21, 2024)：交替使用 Transformer 和 Mamba 层 (比例约 1:7)，总参数 52B (MoE 架构)。仍然需要少量 Transformer 层
5. **BASED**：线性注意力 + 局部注意力的组合
6. **MiniMax-01**：线性注意力 + 全注意力的混合架构，456B MoE，达到 SOTA 水平

#### 4.1.1 Mamba 的核心机制：输入依赖选择

经典 SSM（如 S4）的状态转移参数是固定的，不依赖于输入内容。这意味着模型对所有 token 一视同仁地更新状态——无法根据内容“选择性记忆”或“选择性遗忘”。

##### Mamba 的关键创新：选择性状态空间

经典 SSM 的离散化状态转移方程为：

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t$$

其中  $\bar{A}, \bar{B}$  是固定参数。

Mamba 的核心改变：让  $\bar{B}, C$  和离散化步长  $\Delta$  成为输入的函数：

$$\bar{B}_t = f_B(x_t), \quad C_t = f_C(x_t), \quad \Delta_t = f_\Delta(x_t)$$

其中  $\Delta_t$  控制“记忆门”：

- $\Delta_t$  大  $\rightarrow$  更多地关注当前输入，“忘记”历史
- $\Delta_t$  小  $\rightarrow$  更多地保留历史状态，“忽略”当前输入

这赋予了模型根据内容动态调整信息流的能力。

##### 为什么输入依赖性能解决关联检索？

考虑关联检索任务：“key1:val1 key2:val2 ... query:key2 answer:?”

固定参数的 SSM 无法“选择性存储”特定键值对——它只能将所有信息平滑地压缩到固定大小的状态中。而 Mamba 可以根据 token 内容决定：遇到键值对时增大  $\Delta$  以写入状态，遇到无关 token 时减小  $\Delta$  以保护已存储的信息。

然而，输入依赖性带来了一个工程问题：经典 SSM 可以用快速卷积实现（因为参数固定），但 Mamba 的参数随输入变化，无法直接使用卷积。Mamba 论文的另一个重要贡献是设计了一种**硬件感知的并行扫描算法**（hardware-aware parallel scan），能够在 GPU 上高效实现输入依赖的状态更新。

#### 4.1.2 Jamba：混合架构的设计权衡

纯 Mamba 架构在 1B 规模上匹配 Transformer，但在更大规模上仍有差距。Jamba (AI21, 2024) 采取了务实的混合策略：

##### Jamba 的架构设计

- **层构成**：每 8 层中有 1 层 Transformer（全注意力），其余 7 层为 Mamba
- **MoE**：在 MLP 层中使用 Mixture-of-Experts（总参数 52B，活跃参数 12B）
- **上下文长度**：支持 256K token



### 为什么不能完全去掉 Transformer 层？

目前的证据表明，纯 SSM 架构在以下任务上仍弱于 Transformer：

- **精确信息检索**：从长上下文中提取特定事实
- **上下文学习** (In-context Learning)：从少量示例中学习新模式
- **复杂推理链**：需要精确维护多步中间结果

少量全注意力层提供了“精确查找”能力，弥补了 SSM 的短板。混合架构是当前最务实的平衡点。

Jamba 的推理效率优势显著：相比同参数量的纯 Transformer 模型，KV Cache 缩减约 8 倍（因为只有 1/8 的层需要全注意力缓存），256K 上下文时内存节省尤为明显。

### SSM 对推理的意义

SSM 将  $O(T)$  的 KV Cache 替换为  $O(1)$  的固定状态——这从根本上改变了内存瓶颈。当前的趋势是混合架构（少量全注意力层 + 大量线性/局部注意力层），在保持精度的同时大幅提升推理效率。

## 4.2 扩散模型

扩散模型是另一个有前景的方向，其核心思想是**并行生成所有 token**（而非自回归逐个生成）。

- 传统方法：从随机噪声开始，通过多步迭代精炼生成整个序列
- 扩散模型在图像生成中已非常成功，但在文本生成中还不够成熟
- Inception Labs 的最新结果显示，扩散语言模型在代码生成基准测试上速度远超自回归模型

### 扩散模型为什么能加速推理？

自回归生成的根本瓶颈是顺序依赖——每个 token 依赖前一个 token。扩散模型打破了这一约束，可以并行生成所有位置的 token，然后通过多步精炼逐渐提高质量。虽然需要多次迭代，但每次迭代可以充分利用 GPU 的并行计算能力。

### 扩散语言模型的关键挑战

将扩散模型从连续域（图像像素）迁移到离散域（文本 token）面临独特困难：

- **离散性**：文本 token 是离散的，不能像像素一样直接加高斯噪声。解决方案包括：在 embedding 空间做扩散、使用离散扩散过程（masking/absorbing）
- **长度可变**：文本长度不固定，需要额外的长度预测机制或使用 padding
- **精确性要求**：代码、数学等任务要求 token 级别的精确输出，容错空间极小

### 扩散模型 vs. 自回归模型的 Roofline 分析

假设生成  $T$  个 token，扩散模型需要  $R$  步精炼（典型  $R = 8-32$ ），每步处理整个序列：

- **自回归模型**： $T$  次前向传播，每次  $B \cdot 1$  token  $\rightarrow$  内存受限
- **扩散模型**： $R$  次前向传播，每次  $B \cdot T$  token  $\rightarrow$  计算受限（ $R \ll T$  时）

扩散模型将推理从内存受限区域“推到”计算受限区域，从而更好地利用 GPU 的算力。代价是每步的计算量更大，但总步数大幅减少。

## 4.3 本章小结

替代架构的意义在于**绕过** Transformer 推理的根本限制：

- SSM： $O(1)$  固定状态替代  $O(T)$  KV Cache，彻底消除缓存瓶颈
- 扩散模型：并行生成替代自回归生成，充分利用 GPU 并行计算
- 当前 SOTA 采用混合架构（少量全注意力 + 大量高效层）
- 架构创新可能带来比系统优化更大的推理加速

## 5 量化与模型剪枝

### 5.1 量化：降低数值精度

量化的核心思想极其简单：用更少的位数表示数值，从而减少内存占用和传输量。

#### 常见数值格式

- **fp32**（4 字节）：训练时用于参数和优化器状态
- **bf16**（2 字节）：推理的默认格式
- **fp8**（1 字节）：范围  $[-240, 240]$  (e4m3)，H100 原生支持
- **int8**（1 字节）：范围  $[-128, 127]$ ，精度低于 fp8，但更便宜
- **int4**（0.5 字节）：范围  $[-8, 7]$ ，极低精度

两种量化策略：

- **量化感知训练 (QAT)**：训练过程中模拟量化效果，但需要重新训练，扩展性差
- **训练后量化 (PTQ)**：对已训练好的模型，在少量校准数据上确定量化参数（缩放因子和零点）

| 格式         | 位宽     | 内存节省  | 精度损失 | 硬件支持  | 典型用途    |
|------------|--------|-------|------|-------|---------|
| fp32       | 32 bit | 基线    | 无    | 全部    | 训练（主权重） |
| bf16       | 16 bit | 2×    | 极小   | A100+ | 推理默认    |
| fp8 (e4m3) | 8 bit  | 4×    | 小    | H100+ | 高精度推理   |
| int8       | 8 bit  | 4×    | 小-中  | 广泛    | 通用量化    |
| int4       | 4 bit  | 8×    | 中等   | 有限    | 边缘部署    |
| int3       | 3 bit  | 10.7× | 较大   | 软件模拟  | 极端压缩    |

表 4: 常见数值格式的精度-效率权衡。内存节省相对于 fp32 计算。实际加速比还取决于硬件对该格式的原生支持程度。

量化不是免费的午餐

量化的内存节省是确定的（位宽直接决定），但速度提升取决于硬件支持。例如 int4 在没有原生 int4 Tensor Core 的 GPU 上需要解包（dequantize）后再计算，实际加速可能远低于理论 8×。此外，量化到 4 bit 以下时，模型的“长尾知识”（罕见事实、复杂推理）会显著退化。

5.1.1 LLM.int8(): 处理异常值

标准的 int8 量化步骤：将向量中的值缩放到  $[-128, 127]$  范围内（除以最大绝对值，乘以 128）。

异常值问题

大模型中某些激活值会出现**异常大的数值**（outliers），这些异常值会严重扭曲量化的缩放因子，导致其他正常值的精度急剧下降。这个问题在较大的网络中尤为突出。

LLM.int8() 的解决方案：

- 1. 识别矩阵中的异常值列（通常只占 0.1%）
- 2. 异常值部分保持 fp16 精度处理
- 3. 其余部分使用 int8 量化
- 4. 两部分结果合并

实验表明该方法效果良好（BLOOM 176B 模型上几乎无精度损失），但因为混合精度处理，实际速度比纯 fp16 慢 15-23%。主要优势在于**能将大模型装入更少的 GPU**。

5.1.2 AWQ: 激活感知量化

AWQ（Activation-aware Weight Quantization）的思路：

AWQ 的核心思想

不是所有权重同等重要。通过观察**激活值**来确定哪些权重对输出影响最大，将最重要的 0.1-1% 权重保持高精度，其余权重激进量化。  
AWQ 可以将模型从 fp16 量化到 int3/int4，实现 4 倍内存缩减和 3.2 倍推理加速。

## 5.2 模型剪枝与知识蒸馏

剪枝的思想更为直接：直接移除模型中不重要的部分，然后通过知识蒸馏修复精度损失。

### 剪枝-蒸馏流程 (NVIDIA 方案)

1. 在小规模校准数据集（约 1024 个样本）上，识别重要的层、头和隐藏维度
2. 移除不重要的部分，得到更小的模型
3. 用原始模型作为教师，对剪枝后的模型进行知识蒸馏

### 从头训练 vs. 蒸馏

获得更快推理模型有两条路径：

- **从头训练**：定义更快的架构 → 训练新模型。缺点：训练成本高
- **蒸馏**：定义更快的架构 → 从原始模型初始化权重 → 用蒸馏修复。优点：可以利用已有模型

两种方法的目标相同：在不显著损失精度的前提下，降低推理复杂度。

## 5.3 本章小结

- 量化通过降低数值精度直接减少内存传输量，是最简单直接的推理优化手段
- 异常值处理 (LLM.int8()) 和激活感知量化 (AWQ) 解决了朴素量化的精度问题
- 模型剪枝 + 蒸馏可以在结构层面缩小模型，效果与量化互补
- 这些方法都是“有损”的——需要在精度和效率之间做权衡

## 6 无损加速：投机采样

前面的方法都在“走捷径”——通过降低精度或改变架构来换取速度，可能影响输出质量。投机采样 (Speculative Sampling) 提供了一种**无损**的加速方案：保证从目标模型的精确分布中采样。

### 6.1 核心直觉：验证比生成快

#### 推理两阶段的不对称性

- **Prefill**：给定序列，并行编码所有 token（计算受限，很快）——这本质上就是“验证”
- **Generation**：逐个生成 token（内存受限，很慢）

也就是说：检查一个序列是否合理，比从头生成这个序列要快得多。

## 6.2 投机采样算法

### 投机采样的工作流程

1. 使用一个廉价的草稿模型 (draft model)  $p$  快速生成若干候选 token (如 4 个)
2. 将这些候选 token 送入目标模型 (target model)  $q$  进行并行验证 (Prefill 操作)
3. 使用改进的拒绝采样来决定接受或拒绝每个候选 token
4. 被接受的 token 直接使用；被拒绝时，从修正后的分布中采样一个 token

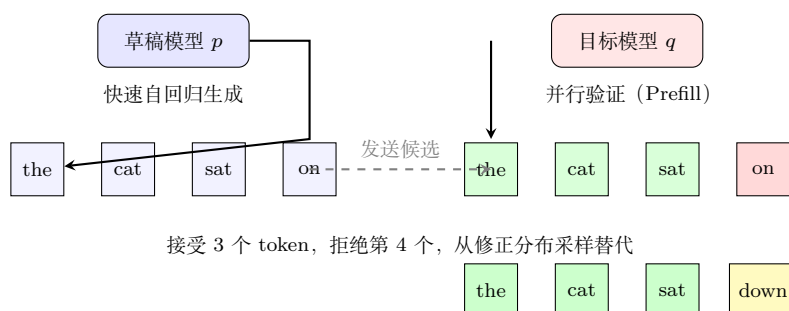


图 7: 投机采样流程: 草稿模型快速猜测 4 个 token, 目标模型并行验证, 接受前 3 个, 对第 4 个从修正分布重采样。

## 6.3 正确性保证

投机采样的关键性质: **保证是目标模型的精确采样。**

以二元词表  $\{A, B\}$  为例进行证明:

设目标模型概率为  $[q(A), q(B)]$ , 草稿模型概率为  $[p(A), p(B)]$ 。假设  $p(A) > q(A)$  (草稿模型过度采样  $A$ )，则  $p(B) < q(B)$ 。

1. 草稿模型提议 token  $x$ , 以概率  $\min(1, q(x)/p(x))$  接受
2. 若接受, 使用  $x$
3. 若拒绝, 从残差分布  $\max(q - p, 0)$  (归一化后) 中采样

计算总采样概率:

$$P[\text{采样}A] = p(A) \cdot \frac{q(A)}{p(A)} + p(B) \cdot 1 \cdot 0 = q(A)$$

$$P[\text{采样}B] = p(B) \cdot 1 + p(A) \cdot \left(1 - \frac{q(A)}{p(A)}\right) \cdot 1 = q(B)$$

### 与标准拒绝采样的区别

标准拒绝采样在拒绝后会重新采样 (可能循环很多次)。投机采样的修改是: 拒绝后不重试, 而是直接从残差分布中采样——保证每一步至少产生一个 token。这种“付出代价直接采样”的策略避免了无限循环。

## 6.4 实践配置与扩展

实际部署中的典型配置：

- 目标模型 70B  $\leftrightarrow$  草稿模型 8B
- 目标模型 8B  $\leftrightarrow$  草稿模型 1B
- 草稿模型越接近目标模型，接受率越高，加速比越大
- 可通过知识蒸馏让草稿模型更好地模拟目标模型

### Worked Example: 投机采样的加速比计算

设草稿模型每生成 1 个 token 耗时  $t_d$ ，目标模型每验证一批 ( $\gamma$  个候选 token) 耗时  $t_v$ ，平均接受率为  $\alpha$ 。

**参数设定：**  $\gamma = 4$  (每轮猜 4 个 token)， $\alpha = 0.8$ ， $t_d = 2$  ms/token， $t_v = 30$  ms/批。

**标准自回归** (目标模型直接生成)：每 token 耗时  $\approx 30$  ms。

**投机采样一轮：**

- 草稿模型生成 4 token： $4 \times 2 = 8$  ms
- 目标模型验证：30 ms
- 总耗时：38 ms
- 期望接受 token 数： $\alpha^0 + \alpha^1 + \alpha^2 + \alpha^3 + (1 - \alpha^4) = 1 + 0.8 + 0.64 + 0.512 + 0.5904 \approx 3.54$  个

**加速比：**  $\frac{3.54 \times 30}{38} \approx 2.8 \times$

注意：如果接受率降到  $\alpha = 0.5$ ，期望接受仅  $\sim 1.94$  个 token，加速比降到  $\frac{1.94 \times 30}{38} \approx 1.5 \times$ 。

**接受率是决定加速效果的关键因素。**

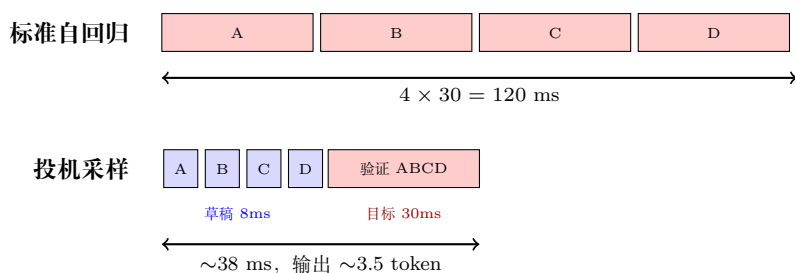


图 8: 标准自回归 vs. 投机采样的时间线对比。投机采样通过草稿模型快速生成 + 目标模型并行验证，大幅缩短生成 4 个 token 的总时间。

实验结果显示约 2 倍的端到端加速。

**改进草稿模型的方向：**

- **Medusa**：草稿模型并行生成多个候选 token (而非自回归)
- **EAGLE**：草稿模型接收目标模型的高层特征，“寄生”在目标模型上生成候选

### 投机采样的适用条件

投机采样的加速效果取决于草稿模型的接受率。如果草稿模型与目标模型的分布差异过大,大量候选 token 会被拒绝,加速效果可能不明显甚至有额外开销。因此,草稿模型的质量是关键。

## 6.5 本章小结

- 投机采样利用了“验证比生成快”的不对称性
- 保证从目标模型的**精确分布**中采样（无损！）
- 典型加速比约 2 倍
- 草稿模型的设计（Medusa、EAGLE 等）是活跃的研究方向
- 可以与前述所有有损方法结合：草稿模型可以使用更激进的量化、更小的架构等

## 7 处理动态工作负载

前面的优化方法关注单个请求或静态批处理的效率。在实际在线服务中,请求是动态到达的,带来了额外的系统级挑战。

### 在线推理服务的挑战

与训练时拥有整齐的 token 块 ( $\text{batch size} \times \text{sequence length}$ ) 不同,在线推理面临三个问题:

1. 请求到达时间不同（等待凑批会伤害先到的请求）
2. 部分序列共享前缀（如系统提示、同一 prompt 的多次采样）
3. 序列长度各不相同（填充 padding 浪费资源）

### 7.1 连续批处理 (Continuous Batching)

#### 连续批处理的核心思想

不再等待一个批次全部完成后才开始下一个批次。而是:

1. 逐步解码,每生成一个 token 后将控制权交还给调度器
2. 新到达的请求可以**立即加入**当前批次
3. 完成生成的请求立即退出,为新请求腾出位置

这种“迭代级调度” (iteration-level scheduling) 来自 Orca 系统。

### Worked Example: 连续批处理的效率提升

假设 3 个请求分别需要生成 10、50、100 个 token，GPU 最大批大小为 3。

**静态批处理：**所有请求必须等最长的（100 步）完成。请求 1 在第 10 步完成后空等 90 步——GPU 利用率仅  $(10 + 50 + 100)/(3 \times 100) \approx 53\%$ 。

**连续批处理：**请求 1 在第 10 步完成后立即退出，新请求 4 立即加入。GPU 始终满载，利用率接近 100%。

在请求长度差异大的实际场景中，连续批处理可以将吞吐量提升 2-3 倍。

**选择性批处理** (Selective Batching) 解决不同长度序列的批处理问题：

- **注意力计算：**每个序列必须单独处理（因为 KV Cache 长度不同）
- **MLP 计算：**所有序列的 token 可以拼接在一起——将  $[3, H]$ ,  $[9, H]$ ,  $[5, H]$  拼为  $[17, H]$ ，因为 MLP 对每个 token 独立作用

## 7.2 PagedAttention 与 vLLM

PagedAttention 是 vLLM 系统的核心创新，解决了 KV Cache 的**内存碎片化**问题。

**传统方式的问题：**

- 请求到达时，为其分配最大可能长度的 KV Cache 空间
- **内部碎片：**实际生成的 token 数远少于预分配空间
- **外部碎片：**不同请求之间有未使用的内存间隙

### PagedAttention 的核心思想

借鉴操作系统的**虚拟内存分页**机制：

1. 将 KV Cache 划分为固定大小的**页** (block)
2. 页可以在物理内存中**不连续**存储
3. 使用页表 (block table) 维护逻辑地址到物理地址的映射
4. 按需分配页，不预分配

传统分配方式



PagedAttention



图 9: 传统连续分配 vs. PagedAttention: 页式存储消除了内部和外部碎片。

PagedAttention 还支持高效的**前缀共享**：



- 多个请求共享相同的系统提示——共享对应的 KV Cache 页
- 同一 prompt 生成多个响应——共享 prompt 部分的页
- 使用**写时复制** (Copy-on-Write)：当共享页需要被修改时才复制

### vLLM 的其他优化

除 PagedAttention 外，vLLM 还包含多项系统优化：

- 融合 kernel：将块读取和注意力计算合并为一个 CUDA kernel，减少启动开销
- 使用最新的计算 kernel (FlashAttention、FlashDecoding)
- CUDA Graphs：预编译执行图，减少 kernel 启动开销

## 7.3 本章小结

- 在线推理服务面临动态、异构的请求流
- 连续批处理消除了等待凑批的延迟浪费
- 选择性批处理允许不同长度序列共存于同一批次
- PagedAttention 将操作系统的内存管理思想应用于 KV Cache，消除碎片化
- 前缀共享和写时复制进一步提高了内存利用率

## 8 总结与延伸

### 8.1 核心要点回顾

本节课从推理的工作负载分析出发，系统性地介绍了优化推理效率的多种技术：

#### 推理优化技术全景

1. **理解工作负载**：推理（特别是 Generation 阶段）是内存受限的，算术强度低
2. **有损优化——缩减 KV Cache**：GQA、MLA、CLA、局部注意力
3. **有损优化——替代架构**：SSM (Mamba 等)、扩散模型
4. **有损优化——量化与剪枝**：int8/int4 量化、AWQ、模型剪枝 + 蒸馏
5. **无损加速**：投机采样（保证精确采样）
6. **系统优化**：连续批处理、PagedAttention

### 8.2 关键洞察

1. **推理不仅仅是“运行模型”**：推理优化本质上是在给定资源预算下最大化输出质量。这意味着架构设计、模型训练和推理部署需要协同考虑。

2. **最大的优化潜力在于架构创新**：系统级优化（更好的 kernel、更高效的调度）能带来常数级的改进。而架构创新（SSM、扩散模型）可以改变计算复杂度的**渐近行为**，潜力远大于系统优化。
3. **借鉴经典计算机科学**：投机采样借鉴了处理器中的**投机执行**（speculative execution），PagedAttention 借鉴了操作系统的**虚拟内存分页**（paging）。

### 8.3 拓展阅读

- **Scaling Book — Transformers 章节**: <https://jax-ml.github.io/scaling-book/transformers/>
- **Scaling Book — Inference 章节**: <https://jax-ml.github.io/scaling-book/inference/>
- **GQA 论文**: Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”
- **MLA / DeepSeek v2**: DeepSeek-AI, “DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model”
- **Mamba**: Gu and Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces” (<https://arxiv.org/abs/2312.00752>)
- **投机采样**: Leviathan et al. (<https://arxiv.org/abs/2211.17192>); Chen et al. (<https://arxiv.org/abs/2302.01318>)
- **Medusa**: <https://arxiv.org/abs/2401.10774>
- **EAGLE**: <https://arxiv.org/abs/2401.15077>
- **LLM.int8()**: Dettmers et al. (<https://arxiv.org/abs/2208.07339>)
- **AWQ**: Lin et al. (<https://arxiv.org/abs/2306.00978>)
- **vLLM / PagedAttention**: Kwon et al. (<https://arxiv.org/abs/2309.06180>)
- **Orca (Continuous Batching)**: Yu et al. (<https://www.usenix.org/system/files/osdi22-yu.pdf>)
- **character.ai 推理优化**: <https://research.character.ai/optimizing-inference/>